LAPORAN PRAKTIKUM

STRUKTUR DATA

"ADT JAM PADA BAHASA PEMROGRAMAN JAVA"

disusun oleh

Hafizh Habibullah

2511531002

Dosen Pengampu:

Dr. Wahyudi, S.T., M.T.

Asisten Praktikum:

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Allah SWT. karena atas rahmat dan karunia-Nya laporan Praktikum Struktur Data ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum pada mata kuliah Struktur Data dengan topik ADT Jam pada Bahasa Pemrograman Java. Melalui laporan ini, penulis berusaha menjelaskan hasil praktikum yang meliputi penjelasan mengenai kode program, tujuan, serta cara kerja program.

Penulis sangat mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan perbaikan di kemudian hari. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, 2 April 2026

Hafizh Habibullah

DAFTAR ISI

KATA PENGANTAR 1
DAFTAR ISI 2
BAB I PENDAHULUAN 4
1.1 Latar Belakang 4
1.2 Tujuan 4
1.3 Manfaat 4
BAB 2 PEMBAHASAN 6
2.1 *Abstract Data Type* (ADT) 6
2.2 Struktur ADT Jam pada Program Java 6
2.2.1 Atribut 7
2.2.2 Konstruktor 7
2.2.3 Validator 7
2.2.4 Selektor (*getter*) 8
2.2.5 Mutator (*setter*) 8
2.3 Fungsi-fungsi penunjang ADT Jam 8
2.3.1 Konversi 8
2.3.2 Relasional 9
2.3.3 Fungsi Aritmatika 10
2.3.4 Fungsi Durasi 11
2.3.5 Fungsi ToString 11
2.4 Kelas Driver Jam 12
2.5 Kelas Driver Jam Lanjutan 13
2.5.1 Fungsi Pembuat Jam dari Input 13
2.5.2 Blok Program Inisialisasi Dua Objek Jam 13
2.5.3 Blok Program Menampilkan Data 14
2.5.4 Blok program Operasi Rasional (Perbandingan) 14
2.5.5 Blok Operasi Aritmatika 15
2.6 Latihan Membuat ADT Mobil 16
2.6.1 Kelas Tugas1Mobil_2511531002 16
2.6.2 Kelas Tugas1MobilDriver_2511531002 17
BAB III KESIMPULAN 18

3.1 Kesimpulan 18
3.2 Saran Pengembangan 18
DAFTAR PUSTAKA..... 19

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengelolaan data berperan sangat penting agar sistem berjalan dengan rapi dan mudah dikembangkan. Salah satu konsep yang digunakan adalah *Abstract Data Type (ADT),* yaitu cara mendefinisikan tipe data beserta operasinya tanpa harus fokus pada detail implementasinya.

Salah satu contoh sederhana ADT adalah tipe data Jam, yang merepresentasikan waktu dalam satu hari dengan tiga komponen: jam, menit, dan detik. Konsep ini banyak digunakan dalam berbagai aplikasi, seperti penjadwalan, pencatatan waktu, dan sistem aktivitas.

Pada praktikum ini dilakukan perancangan dan implementasi ADT Jam menggunakan bahasa Java, sehingga konsep ADT dapat dipahami melalui penerapannya dalam bentuk class pada pemrograman berorientasi objek.

1.2 Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Abstract Data Type (ADT) dalam pemrograman.
2. Mempelajari cara merepresentasikan suatu tipe data dalam bentuk class pada bahasa Java.
3. Mengimplementasikan operasi-operasi dasar pada tipe data Jam, seperti selektor, mutator, serta operasi aritmatika terhadap waktu.
4. Melatih kemampuan dalam merancang program yang terstruktur dan modular.

1.3 Manfaat

Manfaat dari Praktikum ini adalah:

1. Memahami konsep abstraksi data dalam pemrograman.
2. Mengimplementasikan ADT menggunakan bahasa pemrograman Java.
3. Memahami cara melakukan manipulasi data waktu seperti perbandingan waktu, penjumlahan waktu, serta konversi waktu ke dalam satuan detik.

4. Meningkatkan kemampuan dalam merancang program yang lebih terstruktur dan mudah dikembangkan.

BAB 2
PEMBAHASAN

2.1 *Abstract Data Type* (ADT)

ADT (Abstract Data Type) adalah tipe data yang didefinisikan berdasarkan data serta operasi yang dapat dilakukan terhadap data tersebut. Fokus utamanya adalah pada apa yang bisa dilakukan, bukan bagaimana implementasinya.

ADT umumnya memiliki empat jenis method, yaitu konstruktor, validator, selektor, dan mutator. Konstruktor digunakan untuk membuat objek baru. Validator berfungsi untuk memastikan apakah nilai yang diberikan valid. Selektor digunakan untuk mengambil nilai dari objek, sedangkan mutator digunakan untuk mengubah nilai atribut di dalam objek.

Dalam ADT, terdapat dua aspek utama, yaitu spesifikasi dan implementasi. Spesifikasi menjelaskan data yang direpresentasikan, operasi yang tersedia, serta hasil yang diharapkan dari operasi tersebut, tanpa membahas cara kerja internalnya. Contohnya pada jam, yang direpresentasikan oleh tiga komponen utama, yaitu jam, menit, dan detik.

Dalam kehidupan sehari-hari, saat melihat jam, seseorang sebenarnya sedang melakukan operasi untuk memperoleh informasi waktu saat itu. Sementara ketika mengatur ulang waktu pada jam, itu merupakan contoh operasi yang mengubah nilai data yang tersimpan. Hal inilah yang dimaksud dengan operasi dalam ADT.

2.2 Struktur ADT Jam pada Program Java

Terdapat beberapa *method* esensial agar program dapat berjalan dengan tepat. Pada dasarnya, ADT memiliki beberapa operasi dasar, yaitu konstruktor, validator, selektor, dan mutator.

Berikut ini adalah penjelasan *method-method* dasar pada ADT jam:

1. Konstruktor : Membuat objek jam baru.
2. Validator : Memastikan suatu jam memiliki waktu yang sah atau tidak.
3. Selektor : Mengembalikan nilai jam, menit, atau waktu pada jam.
4. Mutator : Mengubah nilai jam, menit, atau waktu pada jam.

2.2.1 Atribut

Atribut utama dalam sebuah jam adalah jam, menit, dan detik yang ditampilkan oleh sebuah jam.

```
public final class Jam_2511531002 {
    private int hh;
    private int mm;
    private int ss;
```

Kode Program 2. 1 - Atribut Utama ADT Jam

2.2.2 Konstruktor

Pada ADT Jam, Konstruktor digunakan untuk menginisialisasi nilai jam, menit, dan detik saat objek pertama kali dibuat. Konstruktor menerima tiga parameter, yaitu h (jam), m (menit), dan s (detik).

```
public Jam_2511531002(int h, int m, int s) {
    this.hh = h; this.mm = m; this.ss = s;
}
```

Kode Program 2. 2 – Potongan Program untuk Konstruktor

2.2.3 Validator

Validator digunakan untuk memastikan apakah sebuah objek dari *class* memiliki atribut-atribut yang sah atau tidak. Untuk ADT Jam, nilai-nilai yang harus diperiksa adalah nilai masing-masing atribut bernilai non-negatif, berada pada rentang 0 sampai 23 untuk jam, dan 0 sampai 59 untuk menit dan detik.

```
public static boolean isValid(int h, int m, int s) {
    return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
}
```

Kode Program 2. 3 – Potongan Program untuk Validator

2.2.4 Selektor (*getter*)

Selektor digunakan untuk menyimpan/mengetahui suatu nilai yang ada pada atribut objek saat diinginkan. Pada ADT jam, nilai yang dikembalikan adalah jam, menit, dan detik dari objek jam tersebut.

```
public int getHour() { return hh; }
public int getMinute() { return mm; }
public int getSecond() { return ss; }
```

Kode Program 2. 4 – Potongan Program untuk Selektor

2.2.5 Mutator (*setter*)

Mutator digunakan untuk mengubah nilai atribut dari suatu objek. Pada ADT Jam, Implementasi mutator digunakan untuk mengubah nilai atribut dari objek jam. Nilai dari jam, menit, dan detik dapat diubah. Mutator pada ADT jam memiliki tiga buat *method* untuk mengubah nilai dari masing-masing atribut utama.

```
public void setHour(int h) {this.hh = h; }
public void setMinute(int m) {this.mm = m; }
public void setSecond(int s) {this.ss = s; }
```

Kode Program 2. 5 - Potongan Program untuk Mutator

2.3 Fungsi-fungsi penunjang ADT Jam

Fungsi-fungsi yang dapat ditambahkan untuk sebuah jam antara lain adalah mengkonversi menjadi satuan detik, membandingkan satu waktu dengan waktu lainnya, menambahkan satuan waktu, dan selisih nilai antara satu waktu dengan waktu lainnya.

2.3.1 Konversi

Untuk mengubah jam ke detik, kalikan nilai jam dengan 3600; dan untuk mengubah menit ke detik, kalikan nilai menit dengan 60. Kemudian, setiap hasil konversi dijumlahkan dan ditambahkan dengan nilai detik yang tersisa.

```
public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
```

Kode Program 2. 6 – Potongan Kode Program untuk Konversi ke Detik

Selain itu, terdapat method fromSeconds(int total) yang digunakan untuk membuat objek Jam berdasarkan jumlah detik yang diberikan. Parameter total menyatakan total detik yang akan dikonversi menjadi satuan jam, menit, dan detik.

```
public static Jam_2511531002 fromSeconds (int total) {
    if (total < 0) throw new IllegalArgumentException("detik negatif");
    total %= 24 * 3600;
    int h = total / 3600; total %= 3600;
    int m = total / 60; int s = total % 60;
    return new Jam_2511531002(h, m, s);
}
```

Kode Program 2. 7 – Potongan Kode Program untuk Membuat Objek Jam dari satuan Detik

Pada awal method dilakukan pemeriksaan apakah nilai detik bernilai negatif atau tidak. Jika iya, maka program akan menghasilkan kesalahan dengan melempar IllegalArgumentException, karena waktu tidak boleh bernilai negatif.

Selanjutnya nilai detik dibatasi dalam rentang waktu satu hari dengna menggunakan operasi modulus terhadap jumlah detik dalam 24 jam. Setelah itu, nilai detik dikonversi menjadi jam, menit, dan detik menggunakan operasi pembagian dan sisa pembagian.

2.3.2 Relasional

Relasional digunakan untuk membandingkan nilai jam satu dan jam lainnya menggunakan method bawaan yaitu compare(). Misalkan terdapat dua variabel bilangan bulat x dan y, compare(int x, int y) membandingkan nilai x dan y. Jika nilai x > y, maka nilai yang dikembalikan adalah positif. Jika nilai x = y, maka nilai yang dikembalikan adalah 0. Jika x < y, maka nilai yang dikembalikan adalah negatif.

```java
public int compareTo (Jam_2511531002 other) {
    return Integer.compare(this.toSeconds(), other.toSeconds());
}
```

Kode Program 2. 8 – Potongan Kode Program untuk Membandingkan Nilai Jam

Program ADT juda menyediakan fungsi khusus untuk membandingkan apakah suatu jam memiliki nilai satuan waktu yang sama. Fungsi ini disebut "equals" dengan parameter bertipe suatu objek, dengan nama variabel "o". Fungsi ini juga memastikan apakah variabel "o" suatu objek dengan kelas Jam sebagai instansinya. Jika tidak, maka nilai kembaliannya otomatis false. Jika iya, maka fungsi ini akan membandingkan kesamaan dari setiap atribut utama dari kedua jam.

```java
public boolean equals (Object o) {
    if (! (o instanceof Jam_2511531002 j)) return false;
    return hh == j.hh && mm == j.mm && ss == j.ss;
}
```

Kode Program 2. 9 – Potongan Kode Program untuk Membandingkan Kesamaan Dua Jam

2.3.3 Fungsi Aritmatika

Fungsi-fungsi aritmatika pada ADT Jam berisikan penjumlahan waktu dalam satuan detik antara kedua jam, selisih jam dalam detik, detik satuan sebelum dan sesudah dari suatu jam, serta beberapa detik sebelum dan sesudah dari suatu jam.

```java
public Jam_2511531002 plus (Jam_2511531002 other) {
    return fromSeconds(this.toSeconds() + other.toSeconds());}
public Jam_2511531002 minus (Jam_2511531002 other) {
    return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600)); }
public Jam_2511531002 nextSecond() {
    return fromSeconds(this.toSeconds() + 1); }
public Jam_2511531002 nextNSeconds(int n) {
    return fromSeconds(this.toSeconds() + Math.max(0, n)); }
public Jam_2511531002 prevSecond() {
    return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600)); }
public Jam_2511531002 prevNSeconds(int n) {
    return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600)); }
```

Kode Program 2. 10 – Potongan-Potongan Program untuk Operasi Aritmatika pada ADT Jam

Baris program pertama mengkonversi satuan waktu yang ada pada jam satu dan lainnya menjadi detik, lalu menjumlahkan hasilnya. Hasil akhirnya akan membuat objek jam baru.

Baris program kedua melakukan hal yang hampir sama dengan baris pertama, bedanya adalah baris program kedua mencari selisih antara satuan detik antara kedua jam, lalu mengkonversinya menjadi objek jam baru.

Baris program ketiga membuat objek jam baru yang menunjukkan waktu satu detik setelah waktu saat ini, dan baris program keempat menghasilkan objek jam baru yang menunjukkan waktu n detik setelah waktu saat ini.

Baris program kelima membuat objek jam baru yang menunjukkan waktu satu detik sebelum waktu saat ini, dan barus program keenam menghasilkan objek jam baru yang menunjukkan waktu n detik sebelum waktu saat ini.

2.3.4 Fungsi Durasi

Fungsi ini mengembalikan durasi antara jam satu dan lainnya. Bedanya dengan fungsi operasi minus sebelumnya adalah fungsi ini mengembalikan nilai dalam satuan detik, bukan dalam bentuk (jam, menit, detik).

```
public static int durasiDetik(Jam_2511531002 jaw, Jam_2511531002 jakh) {
    return jakh.toSeconds() - jaw.toSeconds(); }
```

Kode Program 2. 11 – Potongan Program untuk Mencari Durasi antara Dua Jam

2.3.5 Fungsi ToString

Fungsi ini aslinya adalah Fungsi dari Java yang di-*override*, yang gunanya adalah mengubah suatu angka menjadi String. Tetapi, pada ADT Jam, fungsi ini mengubah angka menjadi format jam.

```
public String toString() {
    return String.format("%02d:%02d:%02d", hh, mm, ss); }
```

Kode Program 2. 12– Potongan Program untuk Mengubah Objek Jam menjadi Format Jam Dalam bentuk String

2.4 Kelas Driver Jam

Kelas Driver adalah kelas yang umum dibuat untuk mengetes ADT yang telah dibuat. Pada kelas JamDriver_2511531002, dibuat beberapa baris program. Pada kelas driver, dilakukan pembuatan beberapa objek jam menggunakan konstruktor, yang kemudian digunakan untuk menguji method-method yang tersedia.

Contoh pengujian yang dilakukan antara lain:

- Membuat objek jam
- Menampilkan objek jam yang telah dibuat
- Menjumlahkan dua jam
- Menambahkan jam sebanyak 20 detik
- Mencari jarak antara dua jam dalam satuan detik
- Membandingkan dua jam

```java
package pekan1_2511531002;

public class JamDriver_2511531002 {
    public static void main(String[] args) {
        Jam_2511531002 a = new Jam_2511531002(23, 59, 50);
        Jam_2511531002 b = new Jam_2511531002(0, 0, 15);
        System.out.println("a        = " + a);
        System.out.println("b        = " + b);
        System.out.println("a+b      = " + a.plus(b));
        System.out.println("next 20s = " + a.nextNSeconds(20));
        System.out.println("durasi(a,b)    = " + Jam_2511531002.durasiDetik(a, b));
        System.out.println("a.compareTo(b) = " + a.compareTo(b));
    }
}
```

Kode Program 2.12 – Kelas JamDriver_2511531002 Sebagai Pengujian

Keluaran dari Program diatas adalah sebagai berikut:

```
<terminated> JamDriver_2511531002 [J
a        = 23:59:50
b        = 00:00:15
a+b      = 00:00:05
next 20s = 00:00:10
durasi(a,b)    = -86375
a.compareTo(b) = 1
```

Gambar 2.1 – Keluaran dari Kelas JamDriver_2511531002

2.5 Kelas Driver Jam Lanjutan

Kelas Driver ini lebih fleksibel dan dinamis karena dapat menerima input dari pengguna. Nama kelas ini adalah JamDriver2_2511531002

2.5.1 Fungsi Pembuat Jam dari Input

```java
private static Jam_2511531002 buatJamDariInput(Scanner sc) {
    int h, m, s;
    while (true) {
        System.out.print("Masukkan Jam (0-23)    : ");
        h = sc.nextInt();
        System.out.print("Masukkan Menit (0-59)  : ");
        m = sc.nextInt();
        System.out.print("Masukkan Detik (0-59)  : ");
        s = sc.nextInt();

        if (Jam_2511531002.isValid(h,  m, s)) {
            return new Jam_2511531002(h, m, s);
        } else {
            System.out.println("[Error] Input tidak valid! Silakan ulangi.\n");
        }
    }
```

Kode Program 2. 13 – Fungsi untuk membuat Jam

Fungsi di atas akan menerima input sebagai parameter dengan nama variabel sc, kemudian menginisialisasi tiga variabel: h, m, dan s.

Input pertama meminta jam dari objek jam, kemudian menitnya, lalu terakhir detiknya. Kemudian perlu di cek apakah jam yang baru dibuat melalui fungsi statis dari kelas Jam_2511531002 sah atau tidak.

2.5.2 Blok Program Inisialisasi Dua Objek Jam

```java
public class JamDriver2_2511531002 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek Jam ===");

        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511531002 j1 = buatJamDariInput(input);

        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511531002 j2 = buatJamDariInput(input);
```

Kode Program 2. 14 – Potongan Program untuk Konstruktor

Kode program di atas akan menginisialisasi objek Jam baru dengan nama j1 dan j2, dan menerima input melalui fungsi buatJamDariInput(). Hasil dari potongan kode Program diatas adalah sebagai beriut jika diberikan input 19,20,15 untuk jam pertama dan 5,30,20 untuk jam kedua.

```
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23)     : 19
Masukkan Menit (0-59)   : 20
Masukkan Detik (0-59)   : 15

--- Input Jam 2 ---
Masukkan Jam (0-23)     : 5
Masukkan Menit (0-59)   : 30
Masukkan Detik (0-59)   : 20
```

Gambar 2.2 – Keluaran dari Potongan Program Inisialisasi

2.5.3 Blok Program Menampilkan Data

```
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)         : " + j1.toString());
        System.out.println("Jam 2 (String)         : " + j2.toString());
        System.out.println("Jam 1 dalam detik      : " + j1.toSeconds());
        System.out.println("Jam 2 dalam detik      : " + j2.toSeconds());
```

Kode Program 2. 15 – Potongan Program untuk Menampilkan Data dari Jam

Blok program di atas mengubah kedua objek jam menjadi 2 bentuk, yaitu dalam format jam melalui baris kedua dan ketiga, dan dalam satuan detik melalui baris keempat dan kelima. Hasil dari blok program tersebut jika digunakan input yang sama dengan sebelumnya adalah sebagai berikut:

```
--- Hasil Operasi ---
Jam 1 (String)         : 19:20:15
Jam 2 (String)         : 05:30:20
Jam 1 dalam detik      : 69615
Jam 2 dalam detik      : 19820
```

Gambar 2.3 – Keluaran dari Blok Program Menampilkan Data

2.5.4 Blok program Operasi Rasional (Perbandingan)

```
int perbandingan = j1.compareTo(j2);
if (perbandingan > 0) {
    System.out.println("Status                 : Jam 1 lebih lambat (setelah) Jam 2");
} else if (perbandingan < 0) {
    System.out.println("Status                 : Jam 1 lebih awal (sebelum) Jam 2");
} else {
    System.out.println("Status                 : Jam 1 dan Jam 2 sama persis");
}
```

Kode Program 2. 16 – Blok Program untuk Operasi Perbandingan Jam

Untuk input yang sama dengan sebeluymnya, hasilnya adalah “Jam 1 lebih lambat (setelah) Jam 2”. Ini disebabkan karena jam pertama menunjukkan pukul 19, dan ini jelas lebih lambat dari ja kedua yang menunjukkan pukul 5.

2.5.5 Blok Operasi Aritmatika

```
    System.out.println("Durasi (J1 ke J2)      : " + Jam_2511531002.durasiDetik(j1, j2) + " detik");

    Jam_2511531002 jNext = j1.nextSecond();
    System.out.println("Jam 1 Detik Berikutnya : " + jNext);
    Jam_2511531002 jPrev = j1.prevSecond();
    System.out.println("Jam 1 Detik Sebelumnya : " + jPrev);

    Jam_2511531002 jHasilPlus = j1.plus(j2);
    System.out.println("Hasil J1 + J2          : " + jHasilPlus);
    input.close();
    System.out.println("\nProgram Selesai.");
}
```

Kode Program 2. 17 - Blok Program Operasi Aritmatika

Kode program di atas digunakan untuk melakukan operasi arimatika sederhana seperti penjumlahan dan pengurangan antar objek jam.

Baris program kedua mencari beberapa perbedaan durasi antara jam pertama dan kedua menggunakan fungsi statis durasiDetik dari kelas Jam_2511531002.

Dua baris selanjutnya mengoperasikan dan menampilkan detik selanjutnya dari jam pertama, dan dua baris setelahnya melakukan hal yang sama untuk jam kedua.

Terakhir, dibuat suatu objek jam baru dengan menjumlahkan kedua objek jam. Program pun selesai.

Hasil dari blok operasi aritmatika objek jam tersebut adalah sebagai berikut:

```
Durasi (J1 ke J2)      : -49795 detik
Jam 1 Detik Berikutnya : 19:20:16
Jam 1 Detik Sebelumnya : 19:20:14
Hasil J1 + J2          : 00:50:35

Program Selesai.
```

Gambar 2.4 – Hasil dari Blok Proram Aritmatika Objek Jam

2.6 Latihan Membuat ADT Mobil

2.6.1 Kelas Tugas1Mobil_2511531002

```java
package pekan1_2511531002;

public class Tugas1Mobil_2511531002 {
    private String nama;
    private int tahun;
    private int cc;
    private int harga;
    private String merk;

    //Validator
    private static boolean isValid(String n, int t, int cc, int h, String m) {
        return !(n.isBlank() && !(t < 0) && !(cc < 0) && !(h < 0) && !(m.isBlank()));
    }

    //Konstruktor
    public Tugas1Mobil_2511531002(String nama, int tahun, int cc, int harga, String merk) {
        if(isValid(nama, tahun, cc, harga, merk)) {
            this.nama = nama;
            this.tahun = tahun;
            this.cc = cc;
            this.harga = harga;
            this.merk = merk;
        }
    }

    //Setter
    public void setNama(String nama) { this.nama = nama; }
    public void setTahun(int tahun) { this.tahun = tahun; }
    public void setCc(int cc) { this.cc = cc; }
    public void setHarga(int harga) { this.harga = harga; }
    public void setMerk(String merk) { this.merk = merk; }

```

```java
    //Getter
    public String getNama() { return nama; }
    public int getTahun() { return tahun; }
    public int getCc() { return cc; }
    public int getHarga() { return harga; }
    public String getMerk() { return merk; }

    //Method-Method Tambahan
    private static Tugas1Mobil_2511531002[] cars = new Tugas1Mobil_2511531002[100];
    private static int n = 0;

    //Tambahkan Mobil
    public static void tambahMobil(Tugas1Mobil_2511531002 M) {
        if(n < cars.length)
        cars[n] = M;
        n++;
    }

    //Hapuskan Mobil berdasarkan Index
    public static void hapusMobil(int idx) {
        if(idx >= 0 && idx < n) {
            for(int i = idx; i < n-1; i++) {
                cars[i] = cars[i+1];
            }
            cars[n-1] = null;
            n--;
        }
    }

```

```java
    //Tampilkan Mobil-Mobil
    public static void tampilData() {
        for(int i = 0; i < n; i++) {
            Tugas1Mobil_2511531002 m = cars[i];
            System.out.println(
                m.getNama()+" "+
                m.getMerk()+" "+
                m.getTahun()+" "+
                m.getCc()+"cc "+
                m.getHarga()
            );
        }
    }
```

Kode Program 2. 18 - Kelas Tugas1Mobil_2511531002

2.6.2 Kelas Tugas1MobilDriver_2511531002

```java
package pekan1_2511531002;

public class Tugas1MobilDriver_2511531002 {
    public static void main(String[] args) {
        Tugas1Mobil_2511531002 M1 = new Tugas1Mobil_2511531002("Avanza", 2020, 1300, 150000000, "Toyota");
        Tugas1Mobil_2511531002 M2 = new Tugas1Mobil_2511531002("HR-V", 2025, 1500, 430000000, "Honda");
        Tugas1Mobil_2511531002 M3 = new Tugas1Mobil_2511531002("Corolla", 1996, 1500, 80000000, "Toyota");
        Tugas1Mobil_2511531002 M4 = new Tugas1Mobil_2511531002("Forester", 2015, 2000, 200000000, "Ford");
        Tugas1Mobil_2511531002 M5 = new Tugas1Mobil_2511531002("CR-V", 2024, 1500, 825000000, "Honda");

        Tugas1Mobil_2511531002.tambahMobil(M1);
        Tugas1Mobil_2511531002.tambahMobil(M2);
        Tugas1Mobil_2511531002.tambahMobil(M3);
        Tugas1Mobil_2511531002.tambahMobil(M4);
        Tugas1Mobil_2511531002.tambahMobil(M5);

        System.out.println("=== Daftar Mobil ===");
        Tugas1Mobil_2511531002.tampilData();

        System.out.println("\n=== Hapus Mobil Index 1 ===");
        Tugas1Mobil_2511531002.hapusMobil(3);

        System.out.println("\n=== Daftar Mobil Setelah Dihapus ===");
        Tugas1Mobil_2511531002.tampilData();
    }
}
```

Kode Program 2. 19 - Kelas Tugas1MobilDriver_2511531002

BAB III
KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa ADT (Abstract Data Type) merupakan konsep yang digunakan untuk mendefinisikan tipe data beserta operasi-operasi yang dapat dilakukan tanpa memperhatikan implementasi internalnya. Penerapan ADT Jam dalam bahasa Java membantu memahami bagaimana data waktu direpresentasikan melalui atribut jam, menit, dan detik, serta bagaimana operasi seperti validasi, pengambilan nilai, dan perubahan nilai diterapkan melalui method. Dengan adanya pemisahan antara spesifikasi dan implementasi, program menjadi lebih terstruktur, mudah dipahami, dan lebih fleksibel untuk dikembangkan.

3.2 Saran Pengembangan

Dalam pengembangan ADT, sebaiknya perancangan spesifikasi dilakukan dengan jelas terlebih dahulu agar implementasi dapat berjalan dengan baik dan sesuai kebutuhan. Selain itu, penggunaan validasi pada setiap input sangat penting untuk menjaga konsistensi data. Praktikum juga dapat dikembangkan lebih lanjut dengan menambahkan fitur tambahan seperti operasi penambahan waktu atau pengurangan waktu agar pemahaman terhadap ADT menjadi lebih mendalam.

DAFTAR PUSTAKA

[1] Modul Praktikum Struktur Data.